# Topic #:

# Single Source Shortest Path

## #What is Shortest Path Problem?

=> Shortest path problem is a problem of finding the shortest path(s) between vertices of a given graph.

=> Shortest path between two vertices is a path that has least cost as compared to all other existing paths.

## # Shortest Path Algorithm:

=> Shortest path algorithms are family of algorithms used for solving the shortest path problem.

## # Applications:

=> Shortest path algorithms have a wide range of applications such as in-

- Google Maps
- Road Networks
- Logistics Research

## Types of Shortest Problem:

- Single - Pair SPP
- Single - Source SPP
- Single - destination SPP
- All pairs SPP

# 1) Single-Source Shortest Path Problem:

=> it is a shortest path problem where the shortest path from a given source vertex to all other remaining vertices is computed.

=> Two algorithms used for solving single-source shortest path problem:

- Dijkstra's
- Bell-man Ford

# Dijkstra's Algorithm:

=> very famous greedy algorithm

=> Compute the shortest path from one particular source node to all other remaining nodes of the graph.

Conditions:

- works only for connected graph.
- (works for directed) as well as undirected graph. (Converse of this algorithm)
- The graph edges should be weighted.
- works only for those graphs that do not contain any negative edge.

$w(u,v) \geq 0$

Dijkstra (G, w, s)

Initialize-Single-Source (G, s)

S = ϕ ——— 1

Q = G.V ——— 1

while Q ≠ ϕ ——— V

u = Extract-min(Q) ——— (V) or for sparse O(log V)

S = S ∪ {u}

for each vertex v ∈ G.Adj[u] V

Relax (u, v, w)

} $V^2$

Relax (u, v, w)

if v.d > u.d + w (u, v) ——— 1

v.d = u.d + w(u, v) ——— 1

v.π = u ——— 1

Initialize-Single-Source (G, s)

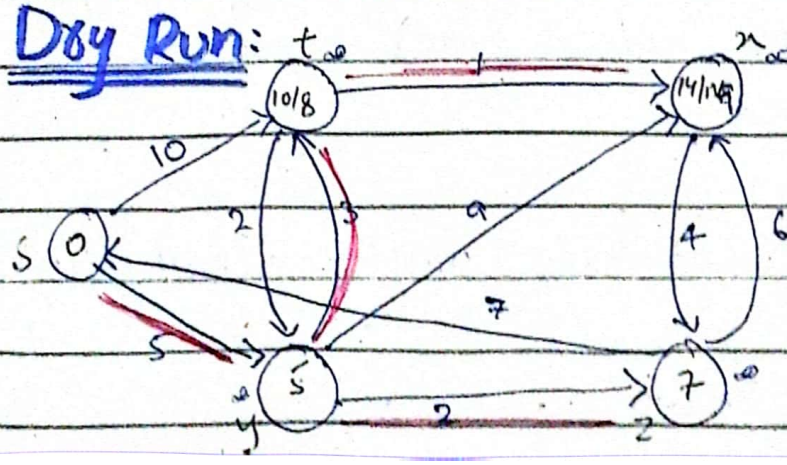
for each vertex v ∈ G.V ——— ∈

v.d = ∞ ——— 1

v.π = NIL ——— 1

s.d = 0 ——— 1

**Dry Run:**



| Node | distance | π | |
|---|---|---|---|
| s | ∞ / 0 | NILL | S = {{s}, {y}, {z}, {t}} |
| t | ∞ / 10 / 8 | NILL/s/y | |
| x | ∞ / 14 / 13 / 9 | NILL/y/z/t | |
| y | ∞ / 5 | NILL/s | |
| z | ∞ / 7 | NILL/y | |

Time complexity : $O(V + E \cdot \log(V))$
Total = $O(E \log V)$

## Applications:

- Used in routing protocols required by the routers to update their forwarding table.
- Provide shortest cost path from the source router to other routers in the network.

## Terms:

S = Source vertex
W = weights
Q = Queue (Priority queue to greedily chooses the nearest node)
S = set

## Main Advantage:

- Little complexity which is almost linear

## Drawbacks:

- unable to handle negative edge

- There is a need to main tracking of vertices, have been visited.
- Does not produce accurate shortest path to the unweighted acyclic graph.
- it does an obscured exploration that consumes a lot of time while processing.

## Bellman Ford Algorithm

=> Solve the single source shortest paths problem.

=> In general in the case in which edge weights may be negative.

=> it works by overestimating the length of the path from the starting vertex to all other vertices.

=> Then it iteratively relaxes those estimates by finding new paths that are shorter than the previously overestimated paths.

=> on a given weighted directed graph $G = (V, E)$ with source $s$ and weight function $w: E \to R$, the bellman ford

algorithm returns a bolean value indicating whether or not a negative-weight cycle that is reachable from the source.

=> If there is such a cycle, the algorithm indicates no solution exists.

if there is no such a cycle, the algorithm produces the shortest paths and their weights.

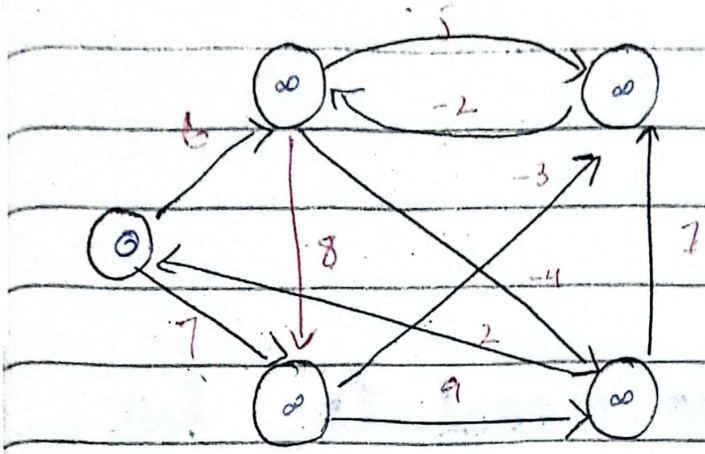
=> This algorithm returns TRue iff the graph contains no negative weight cycles that are reachable from the source.

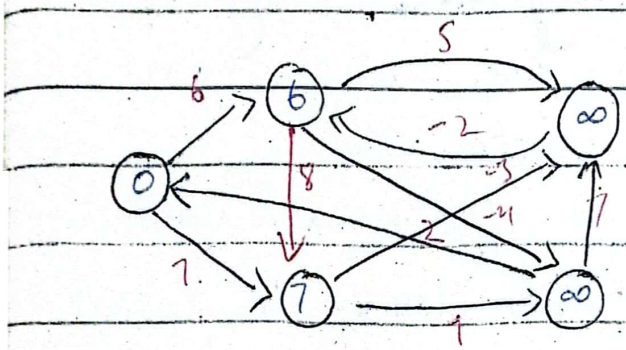
```
BELLMAN-FORD(G,w,s)
  Initialize-Single-Source(G,s)
  for i=1 to |G.V|-1 ---- O(V)
      for each edge (u,v) ∈ G.E
          Relax(u,v,w)
  for each edge (u,v) ∈ G.E ---- O(E)
      if v.d > u.d + w(u,v)
          return FALSE
  return TRUE
```

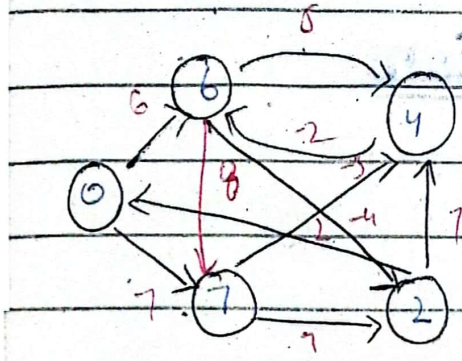
Time complexity O(V·E)

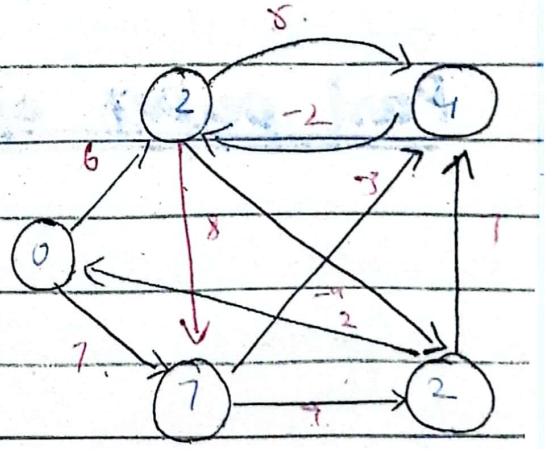
Space complexity O(M)

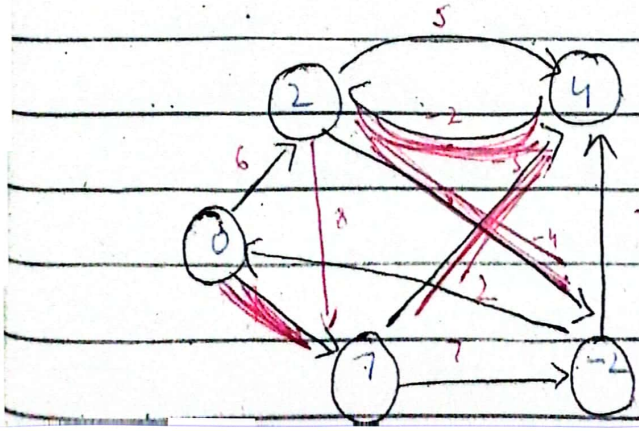
- Note: Loop will execute no of vertices - 1.



| Node | d | π | |
|---|---|---|---|
| s | ∞/0 | NILL | |
| t | ∞/6/2 | NILL/s/x | |
| x | ∞/11/4 | NILL/t/y | Return true |
| y | ∞/7 | NILL/s | |
| z | ∞/2/-2 | NILL/t/t | |

# Why do we need to be careful with negative weights?

=> Negative weight edge can create negative weight cycles.

=> Negative weight cycles can give incorrect result when trying to find out the shortest path.

## Past papers questions

1) what is the purpose of Dijkstra's algorithm?
2) Briefly describe the bellman ford algorithm
3) what is the major difference b/w bell-manford & dijkstra's algorithm?
4) why dijkstra's does not work with negative weight cycle?